



Lesson Plan

HashMaps 1



Today's checklist:

- Introduction to maps and sets
- Types of map
- Difference between ordered and unordered map
- Multimap
- Practice problems

Introduction to Sets

In C++, a set is a container that stores unique elements in no particular order. It is part of the Standard Template Library (STL) and is implemented as a template class in the `<set>`

Definition:

- A set is a container in C++ that stores unique elements in an ordered or sorted sequence.
- It is part of the Standard Template Library (STL).

Uniqueness:

- Sets do not allow duplicate elements; each element is unique within the set.

Ordering:

- Elements in a set are ordered, but the order is not based on the values themselves. Instead, it is determined by the internal representation of the set.

Container Characteristics:

- Sets are typically implemented as binary search trees, providing efficient insertion, deletion, and search operations.

Insertion and Deletion:

- Elements can be inserted into a set, and existing elements are not duplicated. Deletion of elements is also supported.

Search:

- Sets support efficient search operations. You can check whether a specific element is present in the set.

Use Cases:

- Sets are useful when you need to maintain a collection of unique elements and perform operations like insertion, deletion, and search.

Iterating Over Elements:

- You can iterate over the elements of a set using iterators, accessing each element in the ordered sequence.

Common Operations:

- Sets support various common operations like finding an element, checking if an element is present, and iterating through the elements.

Sets are a valuable tool in C++ when you need to work with unique elements and benefit from the ordered nature of the container. They provide efficient algorithms for managing these elements.

STL and important methods in sets

insert():

- **Functionality:** Inserts an element into the set.
- **Syntax:** `mySet.insert(value);`

eg
`std::set<int> mySet;
mySet.insert(42);`

size():

- **Functionality:** Returns the number of elements in the set.
- **Syntax:**

```
std::set<int> mySet = {1, 2, 3, 4, 5};  
size_t setSize = mySet.size(); // setSize will be 5
```

find():

- **Functionality:** Searches for an element in the set.

Return Value:

- **Returns an iterator pointing to the found element if it exists, or `end()` if the element is not present.**

```
std::set<int> mySet = {1, 2, 3, 4, 5};  
auto it = mySet.find(3);  
if (it != mySet.end()) {  
    // Element found  
} else {  
    // Element not found  
}
```

begin():

- **Functionality:** Returns an iterator pointing to the first element in the set.
- **Syntax:**

```
std::set<int> mySet = {1, 2, 3, 4, 5};  
auto firstElement = *(mySet.begin()); // firstElement will be 1
```

end():

- **Functionality:** Returns an iterator pointing just beyond the last element in the set.
- **Syntax:**

```
std::set<int> mySet = {1, 2, 3, 4, 5};  
auto lastElement = *(--mySet.end()); // lastElement will be 5
```

These methods are fundamental when working with sets in C++. They allow you to insert elements, determine the size of the set, search for elements, and iterate through the elements using iterators.

Q. Count Number of Distinct Integers After ReverseOperations [Leetcode - 2442]

Here's a general approach to solving this problem:

Initialize a Set:

- Start by initializing a set to keep track of distinct integers.

Perform Operations:

- Iterate through the array and perform the reverse operation on each element.
- After each reverse operation, update the set with the current state of the array.

Count Distinct Integers:

- After all operations are performed, the size of the set will represent the count of distinct integers.

Here's a sample implementation in C++:

```
static int countDistinctIntegers(const vector<int>& nums) {
    unordered_set<int> seen;
    for (int num : nums) {
        seen.insert(num);
        seen.insert(reverse(num));
    }
    return size(seen);
}
```

- **Time complexity:** $O(n)$
- **Space complexity:** $O(n)$

Q. Find Maximum Number of String Pairs [Leetcode - 2744]

You are given a 0-indexed array of words consisting of distinct strings.

The string `words[i]` can be paired with the string `words[j]` if:

- The string `words[i]` is equal to the reversed string of `words[j]`.
- $0 \leq i < j < \text{words.length}$.

Return the maximum number of pairs that can be formed from the array `words`.

Note that each string can belong in at most one pair.

Example 1:

Input: `words = ["cd","ac","dc","ca","zz"]`

Output: 2

Explanation: In this example, we can form 2 pair of strings in the following way:

- We pair the 0th string with the 2nd string, as the reversed string of word[0] is "dc" and is equal to words[2].
- We pair the 1st string with the 3rd string, as the reversed string of word[1] is "ca" and is equal to words[3].

It can be proven that 2 is the maximum number of pairs that can be formed.

Example 2:

Input: words = ["ab","ba","cc"]

Output: 1

Explanation: In this example, we can form 1 pair of strings in the following way:

- We pair the 0th string with the 1st string, as the reversed string of words[1] is "ab" and is equal to words[0].

It can be proven that 1 is the maximum number of pairs that can be formed.

Example 3:

Input: words = ["aa","ab"]

Output: 0

Explanation: In this example, we are unable to form any pair of strings.

Constraints:

- $1 \leq \text{words.length} \leq 50$
- $\text{words}[i].\text{length} == 2$
- words consists of distinct strings.
- $\text{words}[i]$ contains only lowercase English letters.

Approach:

1. Take a map with seen string as the index and its count as its value.
2. Iterate the words vector and check if its reverse exist in the map or not.
3. If exist in the map then update the ans and decrement the map value as the we have considered this value for pairing.

Code:

```
int maximumNumberOfStringPairs(vector<string>& words) {
    int ans = 0;
    unordered_map<string, int> mp;
    for(auto w: words){
        string r = w;
        reverse(r.begin(), r.end());
        if(mp[r] > 0){ ans++; mp[r]--; }
        else mp[w]++;
    }
    return ans;
}
```

Introduction to Maps

In C++, a map is an associative container that stores elements in a key-value pair format. It is part of the Standard Template Library (STL) and is implemented as the `std::map` class. The key in a map is unique, and each key is associated with a specific value. Maps are also known as associative arrays, dictionaries, or hash maps in other programming languages.

Here's an introduction to maps in C++:

Associative Container:

- A map is an associative container that stores elements in a key-value pair format.
- It allows you to associate a unique key with a specific value.

Key-Value Pairs:

- Each element in a map is a combination of a key and a corresponding value.
- The key is used to uniquely identify an element in the map.
- The value is associated with the key.

Uniqueness of Keys:

- Keys in a map are unique. No two elements in a map can have the same key.

Ordering:

- Elements in a map are ordered based on the key.
- The ordering is either based on the key's comparison operator or a custom comparison function.

Common Operations:

- Insertion: You can insert key-value pairs into a map.
- Access: You can access the value associated with a specific key.
- Iteration: You can iterate over the elements of a map.

Searching and Erasing:

- You can search for an element using its key.
- Erasing an element is done by specifying the key.

Use Cases:

- Maps are useful when you need to maintain a collection of key-value pairs.
- Efficient lookups and retrieval of values based on unique keys are key advantages.

Implementation:

- Internally, maps are implemented as binary search trees or hash tables to provide efficient access.

Applications:

- Maps are commonly used in scenarios where you need to represent relationships between entities with unique identifiers.

Maps are a versatile data structure in C++ that provide efficient access to values based on their unique keys. They are widely used for various applications, offering a balance between simplicity and performance.

STL and important methods in maps

insert():

- **Functionality:** Inserts an element (key-value pair) into the map.
- **Syntax:**

```
myMap.insert(std::make_pair(key, value));  
// or  
myMap[key] = value;
```

- **Note:** If the key already exists, the value associated with the key is updated.

size():

- **Functionality:** Returns the number of elements in the map.
- **Syntax:**

```
size_t mapSize = myMap.size();
```

Example:

```
std::map<int, std::string> myMap = {{1, "One"}, {2, "Two"}};  
size_t mapSize = myMap.size(); // mapSize will be 2
```

find():

- **Functionality:** Searches for an element with a specified key.
- **Syntax:**

```
auto it = myMap.find(key);
```

Return Value:

If the key is found, it points to the element; otherwise, it points to `myMap.end()`.

erase():

- **Functionality:** Removes an element from the map with the specified key.
- **Syntax:**

```
myMap.erase(key);
```

Example:

```
std::map<int, std::string> myMap = {{1, "One"}, {2, "Two"}};  
myMap.erase(1); // Removes the element with key 1
```

count():

- **Functionality:** Returns the number of elements with a specified key (0 or 1).
- **Syntax:**

```
int occurrences = myMap.count(key);
```

Example:

```
std::map<int, std::string> myMap = {{1, "One"}, {2, "Two"}};  
int occurrences = myMap.count(1); // occurrences will be 1
```

These methods are fundamental when working with maps in C++. They allow you to insert elements, determine the size of the map, search for elements, erase elements, and check the number of occurrences of a specific key.

Q. Valid Anagram [Leetcode - 242]

Given two strings *s* and *t*, return true if *t* is an anagram of *s*, and false otherwise.

An Anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

Input: *s* = "anagram", *t* = "nagaram"

Output: true

Example 2:

Input: *s* = "rat", *t* = "car"

Output: false

Constraints:

- **$1 \leq s.length, t.length \leq 5 * 10^4$**
- ***s* and *t* consist of lowercase English letters.**

Explanation:

1. Create an unordered map *count* to store the character frequencies. The key of the map represents a character, and the value represents its frequency.
2. Iterate over each character *x* in string *s*. For each character, increment its frequency in the *count* map by using the *count[x]++* expression.
3. Iterate over each character *x* in string *t*. For each character, decrement its frequency in the *count* map by using the *count[x]--* expression.
4. Iterate over each pair *x* in the *count* map. Each pair consists of a character and its corresponding frequency. Check if any frequency (*x.second*) is non-zero. If any frequency is non-zero, it means there is a character that appears more times in one string than the other, indicating that the strings are not anagrams. In that case, return false.
5. If all frequencies in the *count* map are zero, it means the strings *s* and *t* have the same characters in the same frequencies, making them anagrams. In this case, the function returns true.

This approach counts the frequency of characters in one string and then adjusts the count by decrementing for the other string. If the strings are anagrams, the frequency of each character will cancel out, resulting in a map with all zero frequencies.

The time complexity of this solution is $O(n)$, where n is the total number of characters in both strings. It iterates over each character once to count the frequencies and then compares the frequencies in the map, making it an efficient solution for the problem.

Code:

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        unordered_map<char, int> count;

        // Count the frequency of characters in string s
        for (auto x : s) {
            count[x]++;
        }

        // Decrement the frequency of characters in string t
        for (auto x : t) {
            count[x]--;
        }

        // Check if any character has non-zero frequency
        for (auto x : count) {
            if (x.second != 0) {
                return false;
            }
        }

        return true;
    }
};
```

Q. Two Sum [Leetcode - 1]

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: nums = [3,2,4], target = 6

Output: [1,2]

Example 3:

Input: nums = [3,3], target = 6

Output: [0,1]

Constraints:

- $2 \leq \text{nums.length} \leq 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$
- Only one valid answer exists.

Approach using a hash table:

1. Create an empty hash table to store elements and their indices.
2. Iterate through the array from left to right.
3. For each element $\text{nums}[i]$, calculate the complement by subtracting it from the target: $\text{complement} = \text{target} - \text{nums}[i]$.
4. Check if the complement exists in the hash table. If it does, we have found a solution.
5. If the complement does not exist in the hash table, add the current element $\text{nums}[i]$ to the hash table with its index as the value.
6. Repeat steps 3-5 until we find a solution or reach the end of the array.
7. If no solution is found, return an empty array or an appropriate indicator.

This approach has a time complexity of $O(n)$ since hash table lookups take constant time on average. It allows us to solve the Two Sum problem efficiently by making just one pass through the array.

Code:

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> numMap;
        int n = nums.size();

        for (int i = 0; i < n; i++) {
            int complement = target - nums[i];
            if (numMap.count(complement)) {
                return {numMap[complement], i};
            }
            numMap[nums[i]] = i;
        }

        return {}; // No solution found
    }
};
```

Q. Unique Number of Occurrences [Leetcode - 1207]

Given an array of integers `arr`, return `true` if the number of occurrences of each value in the array is unique or `false` otherwise.

Example 1:

Input: `arr = [1,2,2,1,1,3]`

Output: `true`

Explanation: The value 1 has 3 occurrences, 2 has 2 and 3 has 1. No two values have the same number of occurrences.

Example 2:

Input: `arr = [1,2]`

Output: `false`

Example 3:

Input: `arr = [-3,0,1,-3,1,1,-3,10,0]`

Output: `true`

Constraints:

- $1 \leq \text{arr.length} \leq 1000$
- $-1000 \leq \text{arr}[i] \leq 1000$

Approach:

1. Frequency Counting:

- The code uses an unordered map (`umap`) to count the occurrences of each unique element in the input vector `arr`.
- For each element (`num`) in `arr`, it increments the corresponding frequency in the `umap`.

2. Checking for Unique Frequencies:

- After counting the frequencies in `umap`, the code iterates through the key-value pairs in `umap` using a range-based for loop.
- For each key-value pair (`i`), where `i.first` is the unique element and `i.second` is its frequency, it attempts to insert the frequency (`i.second`) into an unordered set (`uset`).
- The insert function returns a pair where the second element is a boolean indicating whether the insertion was successful. If the insertion is successful (meaning the frequency is unique), the loop continues to the next key-value pair. If the insertion is not successful (meaning the frequency is not unique), the function returns `false`.

3. Return Result:

- If the loop completes without finding any repeated frequencies, the function returns `true`, indicating that all occurrences are unique.

4. Overall Approach:

- Utilize an unordered map to count the occurrences of each unique element.
- Use an unordered set to keep track of unique frequencies.
- Check for uniqueness during the second loop, returning false if a repeated frequency is encountered.
- Return true if all occurrences are unique.

This approach ensures that each unique element has a unique frequency, and it uses unordered map and set data structures for efficient frequency counting and uniqueness checking. If any frequency is found to be repeated, the function returns false; otherwise, it returns true.

Complexity

- Time complexity: $O(N)$
- Space complexity: $O(M)$

Code:

```
class Solution {
public:
    bool uniqueOccurrences(vector<int>& arr) {
        unordered_map<int,int>umap;
        unordered_set<int>uset;
        for(auto num:arr){
            umap[num]++;
        }
        for(auto i : umap)
        {
            if(!uset.insert(i.second).second) return false;
        }
        return true;
    }
};
```

ordered vs unordered

In C++ STL, `std::set`, `std::map`, and `std::multiset` are implemented as ordered containers, while `std::unordered_set`, `std::unordered_map`, and `std::unordered_multiset` are implemented as unordered containers.

Here's a brief overview of the differences between ordered and unordered containers:

Ordered Containers:

Ordering:

- Elements in ordered containers are arranged in a specific order based on the value of the elements or a custom comparison function.
- The order is determined by the key in case of maps and sets, and by the value in case of multisets.

Performance:

- The time complexity of common operations (insertion, deletion, search) is typically $O(\log n)$ for ordered containers.
- These containers are implemented as binary search trees, providing efficient access and ordered traversal.

Use Cases:

- Ordered containers are useful when you need to maintain a sorted collection of elements.
- Common use cases include maintaining a sorted list of unique elements or mapping keys to values in a sorted order.

Unordered Containers:**Ordering:**

- Elements in unordered containers do not have a specific order; their arrangement is based on the hash function used.
- The order is not guaranteed to be the same across different runs of the program.

Performance:

- The time complexity of common operations (insertion, deletion, search) is typically $O(1)$ on average for unordered containers.
- These containers are implemented as hash tables, providing constant-time access in the average case.

Use Cases:

- Unordered containers are suitable when the order of elements doesn't matter, and you want efficient access to elements based on a key.
- Common use cases include quick lookups, maintaining a set of unique elements, or mapping keys to values without requiring a specific order.

Choosing Between Ordered and Unordered:

- If you need a specific order and want to perform operations efficiently on sorted elements, use ordered containers.
- If order doesn't matter, and you prioritize fast average-case performance for insertions, deletions, and searches, use unordered containers.

The choice between ordered and unordered containers depends on the specific requirements of your use case. Each has its advantages and is suitable for different scenarios.

Q. Finding 3-Digit Even Numbers [Leetcode - 2094]

You are given an integer array `digits`, where each element is a digit. The array may contain duplicates.

You need to find all the unique integers that follow the given requirements:

- The integer consists of the concatenation of three elements from digits in any arbitrary order.
- The integer does not have leading zeros.
- The integer is even.

For example, if the given digits were [1, 2, 3], integers 132 and 312 follow the requirements. Return a sorted array of the unique integers.

Example 1:

Input: digits = [2,1,3,0]

Output: [102,120,130,132,210,230,302,310,312,320]

Explanation: All the possible integers that follow the requirements are in the output array. Notice that there are no odd integers or integers with leading zeros.

Example 2:

Input: digits = [2,2,8,8,2]

Output: [222,228,282,288,822,828,882]

Explanation: The same digit can be used as many times as it appears in digits. In this example, the digit 8 is used twice each time in 288, 828, and 882.

Example 3:

Input: digits = [3,7,5]

Output: []

Explanation: No even integers can be formed using the given digits.

Constraints:

- $3 \leq \text{digits.length} \leq 100$
- $0 \leq \text{digits}[i] \leq 9$

Approach:

- We are storing the occurrences of all the elements of digits in a map.
- Because, the 3 digits even number will always come between 100 and 999, we are traversing through all the even numbers from 100 to 999.
- In the second loop, we are storing occurrences of all the digits of the number in another map.
- Because, if occurrence of any digit in the number is greater than the occurrence of that digit in the digits vector, that's why we are breaking the loop and not pushing that particular number in the result vector.

Code:

```
class Solution {
public:
    vector<int> findEvenNumbers(vector<int>& digits) {
        map<int, int> m1;
        vector<int> result;
        for(auto &it: digits)
            m1[it]++;

        for(int i = 100; i < 999; i+= 2){
            map<int, int> mp;
            int num = i;
            while(num){
                int l = num%10;
                mp[l]++;
                if(mp[l] > m1[l])
                    break;
                num /= 10;
            }
            if(num == 0)
                result.push_back(i);
        }
        return result;
    }
};
```

Q. Finding Pairs with a Certain Sum [Leetcode - 1865]

You are given two integer arrays nums1 and nums2. You are tasked to implement a data structure that supports queries of two types:

1. Add a positive integer to an element of a given index in the array nums2.
2. Count the number of pairs (i, j) such that nums1[i] + nums2[j] equals a given value (0 ≤ i < nums1.length and 0 ≤ j < nums2.length).

Implement the FindSumPairs class:

- FindSumPairs(int[] nums1, int[] nums2) Initializes the FindSumPairs object with two integer arrays nums1 and nums2.
- void add(int index, int val) Adds val to nums2[index], i.e., apply nums2[index] += val.
- int count(int tot) Returns the number of pairs (i, j) such that nums1[i] + nums2[j] == tot.

Example 1:

Input

```
["FindSumPairs", "count", "add", "count", "count", "add", "add", "count"]
[[[1, 1, 2, 2, 2, 3], [1, 4, 5, 2, 5, 4]], [7], [3, 2], [8], [4], [0, 1], [1, 1], [7]]
```

Output

```
[null, 8, null, 2, 1, null, null, 11]
```

Explanation:

```
FindSumPairs findSumPairs = new FindSumPairs([1, 1, 2, 2, 2, 3], [1, 4, 5, 2, 5, 4]);
findSumPairs.count(7); // return 8; pairs (2,2), (3,2), (4,2), (2,4), (3,4), (4,4) make 2 + 5 and pairs (5,1), (5,5) make 3 + 4
findSumPairs.add(3, 2); // now nums2 = [1,4,5,4,5,4]
findSumPairs.count(8); // return 2; pairs (5,2), (5,4) make 3 + 5
findSumPairs.count(4); // return 1; pair (5,0) makes 3 + 1
findSumPairs.add(0, 1); // now nums2 = [2,4,5,4,5,4]
findSumPairs.add(1, 1); // now nums2 = [2,5,5,4,5,4]
findSumPairs.count(7); // return 11; pairs (2,1), (2,2), (2,4), (3,1), (3,2), (3,4), (4,1), (4,2), (4,4) make 2 + 5 and pairs (5,3), (5,5) make 3 + 4
```

Constraints:

- $1 \leq \text{nums1.length} \leq 1000$
- $1 \leq \text{nums2.length} \leq 1e5$
- $1 \leq \text{nums1}[i] \leq 1e9$
- $1 \leq \text{nums2}[i] \leq 1e5$
- $0 \leq \text{index} < \text{nums2.length}$
- $1 \leq \text{val} \leq 1e5$
- $1 \leq \text{tot} \leq 1e9$
- At most 1000 calls are made to add and count each.

Intuition:

The problem involves maintaining two arrays and finding the count of pairs with a specified sum. To optimize the count operation, we can use two hash tables (unordered maps) to store the frequency of elements in the two arrays.

Approach:

1. Initialize two hash tables (s1 and s2) to store the frequency of elements in nums1 and nums2, respectively.
2. Create copies of the input arrays (arr1 and arr2).
3. For the add operation, update the corresponding element in arr2 and adjust the frequency in s2.
4. For the count operation, iterate through arr1 and check how many elements in arr2 can form the specified sum with the current element from arr1.
5. Return the count.

Complexity:

- Time complexity:
 - Initialization: $O(n + m)$, where (n) and (m) are the sizes of nums1 and nums2.
 - Add operation: $O(1)$ since we are updating hash tables.
 - Count operation: $O(n)$ in the worst case.
- Space complexity: $O(n + m)$ for the hash tables and copies of the arrays.

Code:

```
class FindSumPairs {
private:
    // Count of elements in nums1 and nums2
    std::unordered_map<int, int> s1, s2;
    // Original copies of nums1 and nums2
    std::vector<int> arr1, arr2;

public:
    // Constructor to initialize the object with nums1 and nums2
    FindSumPairs(std::vector<int>& nums1, std::vector<int>&
nums2) {
        arr1 = nums1;
        arr2 = nums2;

        // Count the frequency of elements in nums1 and nums2
        for (auto x : nums1)
            s1[x]++;

        for (auto x : nums2)
            s2[x]++;
    }

    // Add val to the element at index in nums2
    void add(int index, int val) {
        // Save the original value at index
        int original = arr2[index];
        // Update the value at index by adding val
        arr2[index] += val;

        // Update the frequency count in s2
        s2[arr2[index]]++;

        // If there was more than one occurrence of original
value in nums2, decrement the count
        if (s2[original] > 1)
            s2[original]--;
        else {
            // If only one occurrence, remove the entry from s2
            s2.erase(original);
        }
    }

    // Count pairs (x, y) where x is from nums1, y is from nums2,
and x + y equals tot
    int count(int tot) {
        int ans = 0;
        for (auto x : arr1) {
            int m = tot - x;
            ans += s2[m];
        }
    }
};
```

```

    }
    return ans;
}
};

/**
 * Your FindSumPairs object will be instantiated and called as
such:
 * FindSumPairs* obj = new FindSumPairs(nums1, nums2);
 * obj->add(index,val);
 * int param_2 = obj->count(tot);
 */

/**
 * Your FindSumPairs object will be instantiated and called as
such:
 * FindSumPairs* obj = new FindSumPairs(nums1, nums2);
 * obj->add(index,val);
 * int param_2 = obj->count(tot);
 */

```

Q. Longest Substring without Repeating Characters[Leetcode - 3]

Given a string *s*, find the length of the longest substring without repeating characters.

Example 1:

Input: *s* = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3.

Example 2:

Input: *s* = "bbbbb"

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: *s* = "pwwkew"

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- s consists of English letters, digits, symbols and spaces.

Code:

```
class Solution {
public: int lengthOfLongestSubstring(string s) {
    unordered_set < char > set;

    int i = 0, j = 0, n = s.size(), ans = 0;

    while (i < n && j < n) {
        if (set.find(s[j]) == set.end()) //If the character does
not in the set
        {
            set.insert(s[j++]); //Insert the character in set and
update the j counter
            ans = max(ans, j - i); //Check if the new distance is
longer than the current answer
        } else {
            set.erase(s[i++]);
            /*If character does exist in the set, ie. it is a
repeated character,
we update the left side counter i, and continue with the
checking for substring. */
        }
    }

    return ans;
}
};
```

Time Complexity: $O(n)$

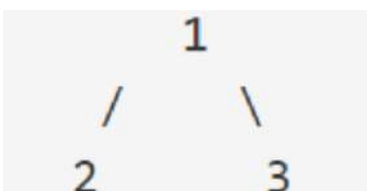
Space Complexity: $O(\min \text{ of } a, b)$ for the unordered set. **a**, is the upper bound of the space complexity.

Top view of binary tree

Problem Statement: Given below is a binary tree. The task is to print the top view of the binary tree. The top view of a binary tree is the set of nodes visible when the tree is viewed from the top.

Example 1:

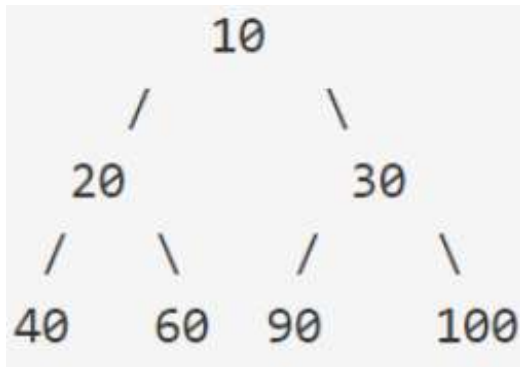
Input:



Output: 2 1 3

Example 2:

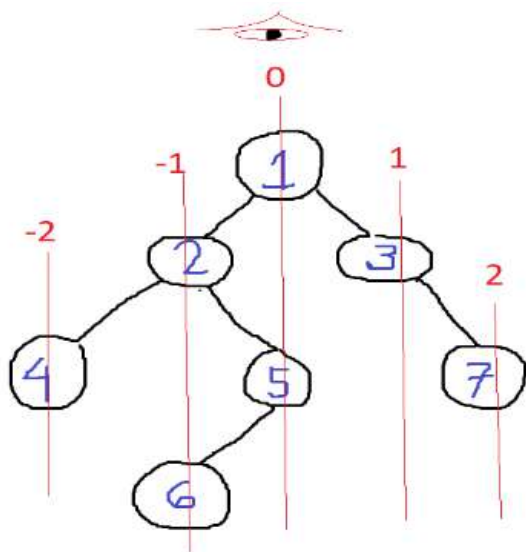
Input:



Output: 40 20 10 30 100

Solution:

Intuition: We can mark straight lines like in the image below and mark them with +ve and -ve indexes. The first node of every line will be my top view.



Approach:

- First we have to make a queue of pairs which have nodes and their respective +ve and -ve indexes.
- Then we need a map data structure to store the lines and the nodes. This map will store the data in the form of sorted orders of keys(Lines).
- Here we will follow the level order traversal.
- Traverse through the nodes starting with root,0 and store them to the queue.
- Until the queue is not empty, store the node and line no. in 2 separate variables .
- Then check if that line is present in the map or not
- If not present then store the line and the node->val to the map
- Otherwise store the node->left and node->right along with there line nos. to the queue.
- Then print the node->val from the map

Code:

```
class Solution
{
    public:
        //Function to return a list of nodes visible from the top
        view
        //from left to right in Binary Tree.
        vector<int> topView(Node *root)
        {
            vector<int> ans;
            if(root == NULL) return ans;
            map<int,int> mpp;
            queue<pair<Node*, int>> q;
            q.push({root, 0});
            while(!q.empty()) {
                auto it = q.front();
                q.pop();
                Node* node = it.first;
                int line = it.second;
                if(mpp.find(line) == mpp.end()) mpp[line] = node
→data;

                if(node->left != NULL) {
                    q.push({node->left, line-1});
                }
                if(node->right != NULL) {
                    q.push({node->right, line + 1});
                }
            }

            for(auto it : mpp) {
                ans.push_back(it.second);
            }
            return ans;
        }
};
```

Time Complexity: $O(N)$

Space Complexity: $O(N)$

Bottom view of binary tree

Problem Statement: Given a binary tree, print the bottom view from left to right. A node is included in the bottom view if it can be seen when we look at the tree from the bottom.

Example 1:

Input:

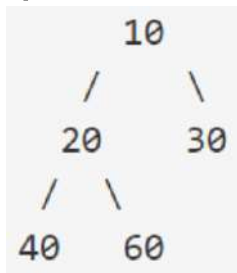


Output: 3 1 2

Explanation: If you look up from the bottom from left to right then first we get 3, then 1 and 2.

Example 2:

Input:

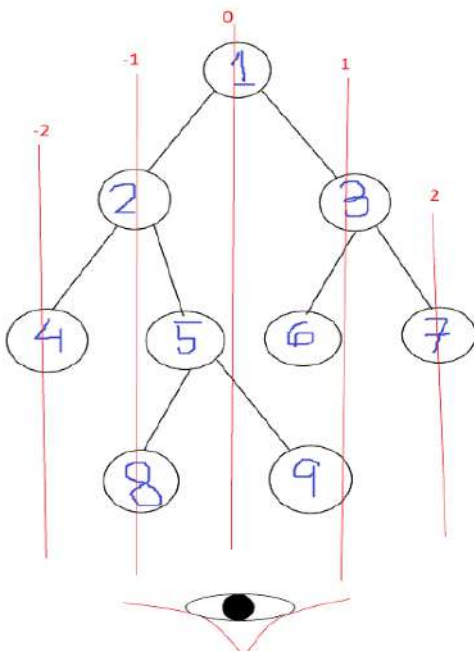


Output: 40 20 60 30

Explanation: If you look up from the bottom from left to right then first we get 40, then 20, 60(it blocks 10), and 30.

Solution:

Intuition: We can mark straight lines like in the image below and mark them with +ve and -ve indexes. The Last node of every line will be my Bottom view.



Approach:

- First we have to make a queue of pair which have nodes and their respective +ve and -ve indexes.
- Then we need a map data structure to store the lines and the nodes. This map will store the data in the form of sorted orders of keys(Lines).
- Here we will follow the level order traversal.
- Traverse through the nodes starting with root,0 and store them to the queue.
- Until the queue is not empty, store the node and line no. in 2 separate variables .
- Then store the line and the node->val to the map, if there will be any node value present that corresponds to a line in the map , it will be replaced by the new node value and by this we will get the last node of each line.
- Store the node->left and node->right along with their line nos. to the queue.
- Then print the node->val from the map

Tip: Here there is only 1 small difference from the Top View of the Tree. Here we don't need to check whether the node is previously present on the map or node before entering it. We have to replace the node of each line if that was previously present on the map.

Code:

```
class Solution {
public:
    vector<int> bottomView(Node *root) {
        vector<int> ans;
        if(root == NULL) return ans;
        map<int,int> mpp;
        queue<pair<Node*, int>> q;
        q.push({root, 0});
        while(!q.empty()) {
            auto it = q.front();
            q.pop();
            Node* node = it.first;
            int line = it.second;
            mpp[line] = node->data;

            if(node->left != NULL) {
                q.push({node->left, line-1});
            }
            if(node->right != NULL) {
                q.push({node->right, line + 1});
            }
        }

        for(auto it : mpp) {
            ans.push_back(it.second);
        }
        return ans;
    }
};
```

Time Complexity: $O(N)$
Space Complexity: $O(N)$

Vertical Order Traversal of a Binary Tree [Leetcode 987]

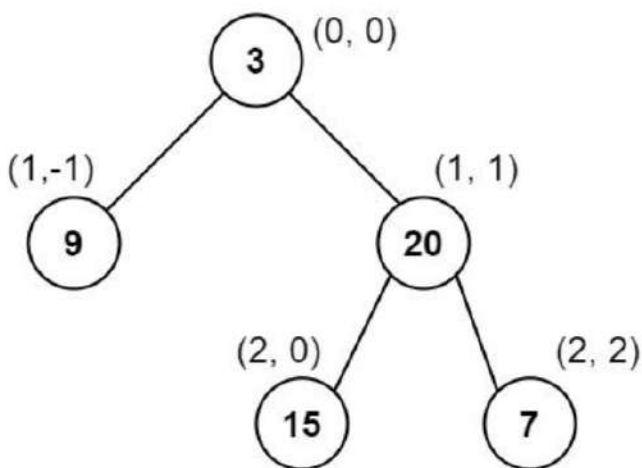
Given the root of a binary tree, calculate the vertical order traversal of the binary tree.

For each node at position (row, col), its left and right children will be at positions (row + 1, col - 1) and (row + 1, col + 1) respectively. The root of the tree is at (0, 0).

The vertical order traversal of a binary tree is a list of top-to-bottom orderings for each column index starting from the leftmost column and ending on the rightmost column. There may be multiple nodes in the same row and same column. In such a case, sort these nodes by their values.

Return the vertical order traversal of the binary tree.

Example 1:



Input: root = [3,9,20,null,null,15,7]

Output: [[9],[3,15],[20],[7]]

Explanation:

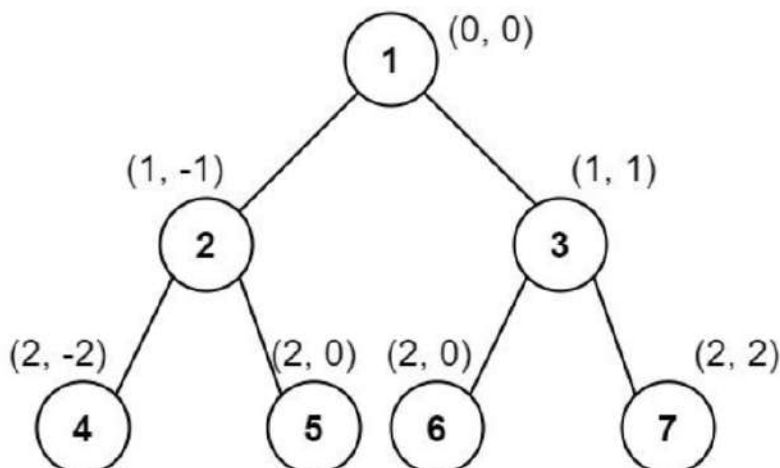
Column -1: Only node 9 is in this column.

Column 0: Nodes 3 and 15 are in this column in that order from top to bottom.

Column 1: Only node 20 is in this column.

Column 2: Only node 7 is in this column.

Example 2:



Input: root = [1,2,3,4,5,6,7]

Output: [[4],[2],[1,5,6],[3],[7]]

Column -2: Only node 4 is in this column.

Column -1: Only node 2 is in this column.

Column 0: Nodes 1, 5, and 6 are in this column.

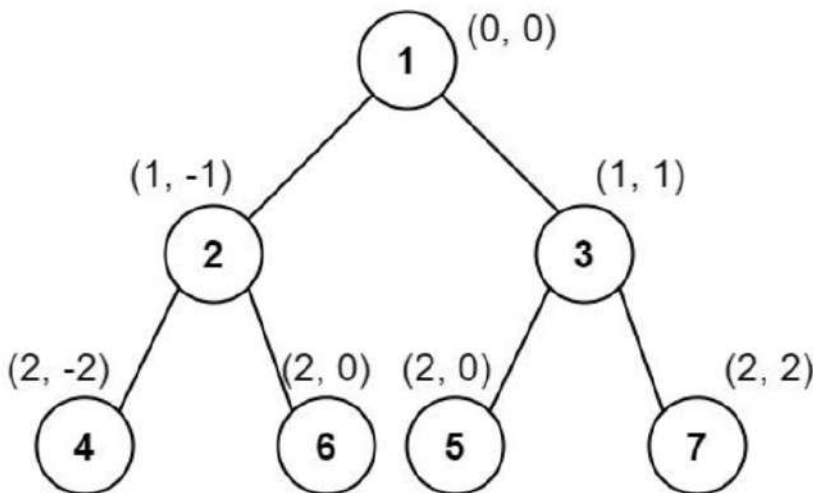
1 is at the top, so it comes first.

5 and 6 are at the same position (2, 0), so we order them by their value, 5 before 6.

Column 1: Only node 3 is in this column.

Column 2: Only node 7 is in this column.

Example 3:



Input: root = [1,2,3,4,6,5,7]

Output: [[4],[2],[1,5,6],[3],[7]]

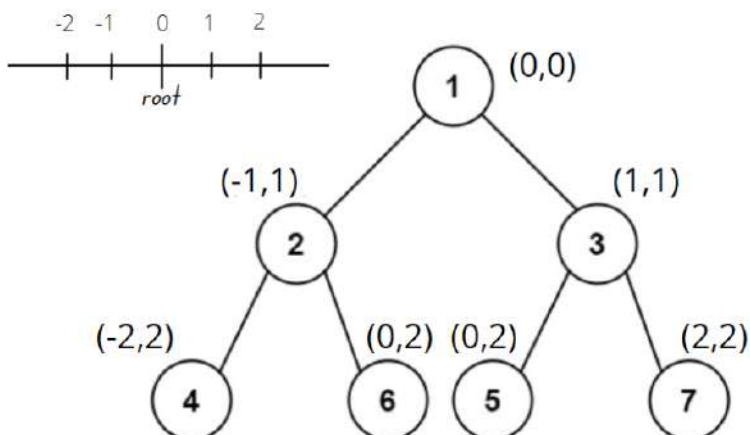
Explanation: This case is the exact same as example 2, but with nodes 5 and 6 swapped.

Note that the solution remains the same since 5 and 6 are in the same location and should be ordered by their values.

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $0 \leq \text{Node.val} \leq 1000$

Let's understand it with the help of an example. Consider the tree given below:



All the nodes are marked with their respective coordinates, we need all this information and then we can apply any traversal. I did level order and so, I stored the node, its x-coordinate, and y-coordinate in a queue. We'll proceed the way we do in level order.

- If left node exists: update $x = x-1$ and $y = y+1$ and push in the queue.
- If right node exists: update $x = x+1$ and $y = y+1$ and push in the queue.

Simultaneously we'll use a map in which we can store x-coordinate, y-coordinate and their corresponding node values. Confusing, right? Don't worry!!

Let's see how will our map look for the above tree:

x	y	node values
-2	2	{4}
-1	1	{2}
0	0	{1}
	2	{5, 6}
1	1	{3}
2	2	{7}

Using this map we'll get our answer.

Implementation of the above approach is as follows:

Code:

```
class Solution {
public:
    vector<vector<int>> verticalTraversal(TreeNode* root) {
        //multiset is used to keep the values in ascending order and
        //we can have repetitive values too
        map<int, map<int, multiset<int>>>mp;
        queue<pair<TreeNode*, pair<int, int>>>q;
        q.push({root, {0, 0}}); //coordinates of the
        //root are (0,0)
        while(!q.empty()){
            TreeNode* node = q.front().first;
            int x = q.front().second.first, y =
            q.front().second.second;
            q.pop();
            mp[x][y].insert(node->val); //inserting values in
            //multiset corresponding to their coordinates
        }
    }
};
```

```

        if(node->left) q.push({node->left, {x-1, y+1}});
        if(node->right) q.push({node->right, {x+1, y+1}});
    }
    vector<vector<int>>>ans;
    for(auto p : mp){                //traversing the map
        vector<int>level;
        for(auto s : p.second){      //p.second is map<int,
multiset<int>>
        //inserting all the values of the multiset in level vector
            level.insert(level.end(), s.second.begin(),
s.second.end());
        }
        ans.push_back(level);
    }
    return ans;
}
};

```

Range frequency queries [Leetcode 2080]

Design a data structure to find the frequency of a given value in a given subarray.

The frequency of a value in a subarray is the number of occurrences of that value in the subarray.

Implement the RangeFreqQuery class:

- RangeFreqQuery(int[] arr) Constructs an instance of the class with the given 0-indexed integer array arr.
- int query(int left, int right, int value) Returns the frequency of value in the subarray arr[left...right].

A subarray is a contiguous sequence of elements within an array. arr[left...right] denotes the subarray that contains the elements of nums between indices left and right (inclusive).

Example 1:

Input

["RangeFreqQuery", "query", "query"]

[[[12, 33, 4, 56, 22, 2, 34, 33, 22, 12, 34, 56]], [1, 2, 4], [0, 11, 33]]

Output

[null, 1, 2]

Explanation:

RangeFreqQuery rangeFreqQuery = new RangeFreqQuery([12, 33, 4, 56, 22, 2, 34, 33, 22, 12, 34, 56]);

rangeFreqQuery.query(1, 2, 4); // return 1. The value 4 occurs 1 time in the subarray [33, 4]

rangeFreqQuery.query(0, 11, 33); // return 2. The value 33 occurs 2 times in the whole array.

Constraints:

- $1 \leq \text{arr.length} \leq 10^5$
- $1 \leq \text{arr}[i], \text{value} \leq 10^4$
- $0 \leq \text{left} \leq \text{right} < \text{arr.length}$
- At most 105 calls will be made to query

Approach:

Make a map of vectors $m<\text{int}, \text{vector}>$. Here key will be elements and vector will hold the indexes of the occurrence of elements in arr. All the vectors in map will be sorted by default (why?). Now we have to return the occurrence of a element in range say $[l, r]$. Use binary search on the $m[\text{val}]$ for l and r . In binary search always return position as greater than or equal to of the passed key.

Suppose in $\text{arr}[1, 2, 3, 5, 3, 6, 3, 2, 3, 2, 3]$

for $\text{val} = 3$ array of indexes of occurrence will be $\text{idxArr} = [2, 4, 6, 8, 9, 11]$

Now we have to find occurrence of 3 in range $[3, 10]$ bs for 3 will return 1 (as $\text{idxArr}[1] > 3$) bs for 10 will return 5 (as $\text{idxArr}[5] > 10$)

answer will be $\rightarrow (5 - 1) = 4$

Complexity:

Time complexity: $O(\log N)$

Space complexity: $O(N)$

Code:

```
class RangeFreqQuery {
public:
    map<int, vector<int>> ma;
    int f;
    int l, r;

    int bs(vector<int> &arr, int s, int e, int len, int val, int *found){
        if(s < 0 || e ≥ len || s > e){
            return s;
        }

        int m = (s + e) / 2;

        if (arr[m] == val){
            *found = 1;
            return m;
        }

        if(arr[m] > val){
            return bs(arr, s, m - 1, len, val, found);
        }

        return bs(arr, m + 1, e, len, val, found);
    }

    RangeFreqQuery(vector<int>& arr) {
        int len = arr.size();
        for(int i = 0; i < len; i++){
            ma[arr[i]].push_back(i);
        }
    }
}
```



```
int query(int left, int right, int value) {
    int len = ma[value].size();
    f = 0;
    l = bs(ma[value], 0, len - 1, len, left, &f);
    f = 0;
    r = bs(ma[value], 0, len - 1, len, right, &f);

    return r - l + f;
}

/**
 * Your RangeFreqQuery object will be instantiated and called as
 * such:
 * RangeFreqQuery* obj = new RangeFreqQuery(arr);
 * int param_1 = obj->query(left,right,value);
 */
```

Count Nice Pairs in an Array [Leetcode 1814]

You are given an array `nums` that consists of non-negative integers. Let us define `rev(x)` as the reverse of the non-negative integer `x`. For example, `rev(123) = 321`, and `rev(120) = 21`. A pair of indices (i, j) is nice if it satisfies all of the following conditions:

- $0 \leq i < j < \text{nums.length}$
- $\text{nums}[i] + \text{rev}(\text{nums}[j]) = \text{nums}[j] + \text{rev}(\text{nums}[i])$

Return the number of nice pairs of indices. Since that number can be too large, return it modulo $10^9 + 7$.

Example 1:

Input: `nums = [42,11,1,97]`

Output: 2

- **Explanation:** The two pairs are:
- $(0,3) : 42 + \text{rev}(97) = 42 + 79 = 121, 97 + \text{rev}(42) = 97 + 24 = 121.$
- $(1,2) : 11 + \text{rev}(1) = 11 + 1 = 12, 1 + \text{rev}(11) = 1 + 11 = 12.$

Example 2:

Input: `nums = [13,10,35,24,76]`

Output: 4

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $0 \leq \text{nums}[i] \leq 10^9$

Approach:

- Define a function reverse to reverse the digits of a non-negative integer.
- Iterate through the array and subtract the reversed form of each number from the original number, updating the array with the differences.
- Sort the modified array to group identical differences together.
- Iterate through the sorted array to count the occurrences of each unique difference.
- Use the count of occurrences to calculate the number of pairs that meet the condition.
- Return the result modulo $10^9 + 7$.

Complexity:

- Time complexity:
 - $O(n \log n)$
 - $O(n \log n)$
- Space complexity:
 - $O(n)$
 - $O(n)$ for the modified array.

Code:

```
class Solution {
public:
    int countNicePairs(std::vector<int>& nums) {
        const int mod = 1000000007;
        int len = nums.size();
        for (int i = 0; i < len; i++) {
            nums[i] = nums[i] - reverse(nums[i]);
        }
        std::sort(nums.begin(), nums.end());
        long res = 0;
        for (int i = 0; i < len - 1; i++) {
            long count = 1;
            while (i < len - 1 && nums[i] == nums[i + 1]) {
                count++;
                i++;
            }
            res = (res % mod + (count * (count - 1)) / 2) % mod;
        }

        return static_cast<int>(res); // Add this line to return
        the result.
    }

private:
    int reverse(int num) {
        int rev = 0;
        while (num > 0) {
            rev = rev * 10 + num % 10;
            num /= 10;
        }
        return rev;
    }
};
```

Unique Length-3 Palindromic Subsequences

[Leetcode 1930]

Given a string *s*, return the number of unique palindromes of length three that are a subsequence of *s*. Note that even if there are multiple ways to obtain the same subsequence, it is still only counted once.

A palindrome is a string that reads the same forwards and backwards.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

Example 1:

Input: *s* = "aabca"

Output: 3

Explanation: The 3 palindromic subsequences of length 3 are:

- "aba" (subsequence of "aabca")
- "aaa" (subsequence of "aabca")
- "aca" (subsequence of "aabca")

Example 2:

Input: *s* = "adc"

Output: 0

Explanation: There are no palindromic subsequences of length 3 in "adc".

Example 3:

Input: *s* = "bbcbaba"

Output: 4

Explanation: The 4 palindromic subsequences of length 3 are:

- "bbb" (subsequence of "bbcbaba")
- "bcb" (subsequence of "bbcbaba")
- "bab" (subsequence of "bbcbaba")
- "aba" (subsequence of "bbcbaba")

Constraints:

- $3 \leq s.length \leq 105$
- *s* consists of only lowercase English letters.

Approach:

1. Initialize Arrays: Create arrays *minExist* and *maxExist* with 26 elements each, initialized to *INT_MAX* and *INT_MIN*, respectively.
2. Populate Arrays: Iterate through each character in the input string, updating the minimum and maximum occurrences for each character in the corresponding arrays.
3. Count Unique Palindromic Subsequences:

- Iterate over each character in the alphabet (26 characters).
 - Check if the character has occurred in the input string. If not, skip to the next character.
 - Create an empty set uniqueCharsBetween to store unique characters between the minimum and maximum occurrences of the current character.
 - Iterate over the characters between the minimum and maximum occurrences, adding each character to the set.
 - Add the count of unique characters between occurrences to uniqueCount.
4. Return Result: Return the total count of unique palindromic subsequences uniqueCount.

Complexity:

- Time complexity:
- $O(N)O(N)$
- $O(N)$

Since we iterating two passes, first pass to compute first and last occurrence for each character with time N and second pass to compute number of unique palindromes. In the second pass we iterate over all the unique characters which are 26 and we iterate between their first and last occurrence to compute the number of unique characters which can be at most N characters. So time complexity is $26 * N + N$ which is $27 * N$ which is also $O(N)$. Where N is number of characters in our input string.

- Space complexity:
- $O(1)O(1)$
- $O(1)$

Since we are storing two arrays to compute last and first occurrence each one of them is of size 26 and we use hash set that can be at maximum with size 26 so the space complexity is still $O(1)$. Since we have constant space variables.

Code:

```
class Solution {
public:
    int countPalindromicSubsequence(string inputString) {
        // Arrays to store the minimum and maximum occurrences of
        // each character in the input string
        vector<int> minExist(26, INT_MAX);
        vector<int> maxExist(26, INT_MIN);

        // Populate minExist and maxExist arrays
        for(int i = 0; i < inputString.size(); i++) {
            int charIndex = inputString[i] - 'a';
            minExist[charIndex] = min(minExist[charIndex], i);
            maxExist[charIndex] = max(maxExist[charIndex], i);
        }

        // Variable to store the final count of unique
        // palindromic subsequences
        int uniqueCount = 0;
    }
};
```

```

        // Iterate over each character in the alphabet
        for (int charIndex = 0; charIndex < 26; charIndex++) {
            // Check if the character has occurred in the input
            string
                if (minExist[charIndex] == INT_MAX ||
maxExist[charIndex] == INT_MIN) {
                    continue; // No occurrences, move to the next
character
                }

            // Set to store unique characters between the minimum
and maximum occurrences
            unordered_set<char> uniqueCharsBetween;

            // Iterate over the characters between the minimum
and maximum occurrences
            for (int j = minExist[charIndex] + 1; j <
maxExist[charIndex]; j++) {
                uniqueCharsBetween.insert(inputString[j]);
            }

            // Add the count of unique characters between the
occurrences to the final count
            uniqueCount += uniqueCharsBetween.size();
        }

        // Return the total count of unique palindromic
subsequences
        return uniqueCount;
    }
};

```

Sum of Beauty of All Substrings [Leetcode 1781]

The beauty of a string is the difference in frequencies between the most frequent and least frequent characters.

- For example, the beauty of "abaacc" is $3 - 1 = 2$.

Given a string *s*, return the sum of beauty of all of its substrings.

Example 1:

Input: *s* = "aabcba"

Output: 5

Explanation: The substrings with non-zero beauty are ["aab", "aabc", "aabcba", "abcb", "bcb"], each with beauty equal to 1.

Example 2:

Input: *s* = "aabcbaa"

Output: 17

Constraints:

- $1 \leq s.length \leq 500$
- s consists of only lowercase English letters.

Approach:

Get all the subarrays using $O(n^2)$ and check max and min for all subarrays and then calculate sum which is equals to max-min and keep in mind that min should be greater than or equal to 1 it can not be zero

Complexity:

- Time complexity:
- $O(n^2)O(n^2)$
- $O(n^2)$
- Space complexity:
- $O(26)O(26)$
- $O(26)$

Code:

```
class Solution {
public:
    int fun2(vector<int>mp){
        int k=INT_MAX;
        for(auto it:mp){
            if(it<=k &&it>=1)k=it;
        }
        return k;
    }
    int fun(vector<int>mp){
        int k=INT_MIN;
        for(auto it:mp){
            if(it>=k)k=it;
        }
        return k;
    }
    int beautySum(string s) {
        int sum=0;
        for(int i=0;i<s.length();i++){
            vector<int>mp(27);
            for(int j=i;j<s.length();j++){
                mp[s[j]-'a']++;
                int mx=fun(mp);
                int mn=fun2(mp);
                sum+=mx-mn;
            }
        }
        return sum;
    }
};
```

Count Substrings That Differ by One Character [Leetcode 1638]

Given two strings s and t , find the number of ways you can choose a non-empty substring of s and replace a single character by a different character such that the resulting substring is a substring of t . In other words, find the number of substrings in s that differ from some substring in t by exactly one character.

For example, the underlined substrings in "computer" and "computation" only differ by the 'e'/'a', so this is a valid way.

Return the number of substrings that satisfy the condition above.

A substring is a contiguous sequence of characters within a string.

Example 1:

Input: $s = \text{"aba"}, t = \text{"baba"}$

Output: 6

Explanation: The following are the pairs of substrings from s and t that differ by exactly 1 character:

("aba", "baba")

("aba", "baba")

("aba", "baba")

("aba", "baba")

("aba", "baba")

("aba", "baba")

The underlined portions are the substrings that are chosen from s and t .

Example 2:

Input: $s = \text{"ab"}, t = \text{"bb"}$

Output: 3

Explanation: The following are the pairs of substrings from s and t that differ by 1 character:

("ab", "bb")

("ab", "bb")

("ab", "bb")

The underlined portions are the substrings that are chosen from s and t .

Constraints:

- $1 \leq s.length, t.length \leq 100$
- s and t consist of lowercase English letters only.

Code:

```
class Solution {
public:
    int countSubstrings(string s, string t) {
        int res = 0 ;
        for (int i = 0; i < s.length(); ++i)
            res += helper(s, t, i, 0);
        for (int j = 1; j < t.length(); ++j)
            res += helper(s, t, 0, j);
    }
};
```

```

        return res;
    }

    int helper(string s, string t, int i, int j) {
        int res = 0, pre = 0, cur = 0;
        for (int n = s.length(), m = t.length(); i < n && j < m;
++i, ++j) {
            cur++;
            if (s[i] != t[j])
                pre = cur, cur = 0;
            res += pre;
        }
        return res;
    }
};

```

Determine if Two Strings Are Close [Leetcode 1657]

Two strings are considered close if you can attain one from the other using the following operations:

- Operation 1: Swap any two existing characters.
 - For example, abcde -> aecdb
- Operation 2: Transform every occurrence of one existing character into another existing character, and do the same with the other character.
 - For example, aacabb -> bbcbba (all a's turn into b's, and all b's turn into a's)

You can use the operations on either string as many times as necessary.

Given two strings, word1 and word2, return true if word1 and word2 are close, and false otherwise.

Example 1:

Input: word1 = "abc", word2 = "bca"

Output: true

Explanation: You can attain word2 from word1 in 2 operations.

Apply Operation 1: "abc" -> "acb"

Apply Operation 1: "acb" -> "bca"

Example 2:

Input: word1 = "a", word2 = "aa"

Output: false

Explanation: It is impossible to attain word2 from word1, or vice versa, in any number of operations.

Example 3:

Input: word1 = "cabbbba", word2 = "abbccc"

Output: true

Explanation: You can attain word2 from word1 in 3 operations.

Apply Operation 1: "cabbbba" -> "caabbbb"

Apply Operation 2: "caabbbb" -> "baacccc"

Apply Operation 2: "baacccc" -> "abbccc"

Constraints:

- $1 \leq \text{word1.length}, \text{word2.length} \leq 105$
- word1 and word2 contain only lowercase English letters.

Approach:

- condition1 : we need the size of both strings to be same
- condition2 : we need freq of char in strings to be same, irrespective of the order

If above 2 conditions are satisfied then just swapping will get us the word2 from word1

Code:

```
class Solution {
public:
    bool closeStrings(string word1, string word2) {
        // condition1 : we need the size of both strings to be
        same
        // condition2 : we need freq of char in strings to be
        same, irrespective of the order
        // if above 2 conditions are satisfied then just swapping
        will get us the word2 from word1

        if(word1.size() != word2.size())
            return false;

        set<char> s1, s2;
        vector<int> freq1(26,0), freq2(26,0);

        for(int i=0; i<word1.size(); i++)
        {
            s1.insert(word1[i]);
            s2.insert(word2[i]);

            freq1[word1[i] - 'a']++;
            freq2[word2[i] - 'a']++;
        }

        sort(freq1.begin(), freq1.end());
        sort(freq2.begin(), freq2.end());

        if(s1==s2 && freq1==freq2)
            return true;
        else
            return false;
    }
};
```

Time Complexity : $O(n)$

Max Number of K-Sum Pairs [Leetcode 1679]

You are given an integer array `nums` and an integer `k`.

In one operation, you can pick two numbers from the array whose sum equals `k` and remove them from the array.

Return the maximum number of operations you can perform on the array.

Example 1:

Input: `nums = [1,2,3,4]`, `k = 5`

Output: 2

Explanation: Starting with `nums = [1,2,3,4]`:

- Remove numbers 1 and 4, then `nums = [2,3]`
- Remove numbers 2 and 3, then `nums = []`

There are no more pairs that sum up to 5, hence a total of 2 operations.

Example 2:

Input: `nums = [3,1,3,4,3]`, `k = 6`

Output: 1

Explanation: Starting with `nums = [3,1,3,4,3]`:

- Remove the first two 3's, then `nums = [1,4,3]`

There are no more pairs that sum up to 6, hence a total of 1 operation.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^9$
- $1 \leq k \leq 10^9$

Approach:

This code implements a solution to find the maximum number of pairs of integers from the `nums` vector whose sum is equal to `k`. The solution uses an unordered map `umap` to keep track of the frequencies of numbers encountered in the `nums` vector. The idea is to find pairs of numbers such that their sum equals `k` by checking if the complement of the current number ($k - \text{nums}[i]$) has been encountered before.

Here's a step-by-step explanation of the solution:

- Initialize an unordered map `umap` to store the frequencies of numbers encountered and an integer `ans` to store the count of pairs whose sum equals `k`.
- Iterate through the `nums` vector using a for loop.
- For each element `nums[i]` in the `nums` vector:
 - Calculate the complement of `nums[i]` with respect to `k` as $k - \text{nums}[i]$. This is the value that, when added to `nums[i]`, would make the sum equal to `k`.
 - Check if the complement ($k - \text{nums}[i]$) is already present in the `umap` and its frequency is greater than zero:

- If it is, decrement the frequency of the complement in umap by 1 (to mark that it has been used), and increment the ans variable by 1. This represents finding a pair whose sum equals k.
- If the complement is not in umap or its frequency is zero, it means we haven't encountered a number that can form a pair with the current nums[i] to make the sum equal to k. In this case, we increment the frequency of nums[i] in umap.
- Continue this process for all elements in the nums vector.
- Finally, return the value of ans, which represents the maximum number of pairs whose sum equals k.

This algorithm efficiently counts the pairs whose sum is equal to k by using the umap to keep track of the numbers encountered and their frequencies.

Complexity:

- **Time complexity:** $O(N)$
- **Space complexity:** $O(N)$

Code:

```
class Solution {
public: int maxOperations(vector < int > & nums, int k) {
    unordered_map < int, int > umap;
    int ans = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (umap[k - nums[i]] > 0) {
            umap[k - nums[i]]--;
            ans++;
        } else {
            umap[nums[i]]++;
        }
    }
    return ans;
};
```

Check If Array Pairs Are Divisible by k [Leetcode 1497]

Given an array of integers arr of even length n and an integer k.

We want to divide the array into exactly $n / 2$ pairs such that the sum of each pair is divisible by k.

Return true If you can find a way to do that or false otherwise.

Example 1:

Input: arr = [1,2,3,4,5,10,6,7,8,9], k = 5

Output: true

Explanation: Pairs are (1,9),(2,8),(3,7),(4,6) and (5,10).

Example 2:

Input: arr = [1,2,3,4,5,6], k = 7

Output: true

Explanation: Pairs are (1,6),(2,5) and (3,4).

Example 3:**Input:** arr = [1,2,3,4,5,6], k = 10**Output:** false**Explanation:** You can try all possible pairs to see that there is no way to divide arr into 3 pairs each with sum divisible by 10.**Constraints:**

- arr.length == n
- $1 \leq n \leq 105$
- n is even.
- $-109 \leq \text{arr}[i] \leq 109$
- $1 \leq k \leq 105$

Approach:

We can iterate through the array and count the number of elements that have the same remainder when divided by the given number k. We can use a vector of size k to store the count of the remainders. We then check if the count of the remainder 0 is odd, since we can't split an odd number of elements into pairs. We then iterate through the vector starting from index 1 to k/2 and check if the count of elements with the same remainder is the same. If all the remainders have an even number of elements, then we can return true.

Complexity:

- Time complexity: $O(n)$
- Space complexity: $O(k)$

Code:

```
class Solution {
public:
    bool canArrange(vector<int>& arr, int k) {
        int n = arr.size();
        vector<int> remainders(k);
        for (int i = 0; i < n; ++i) {
            int remainder = arr[i] % k;
            if (remainder < 0) {
                remainder += k;
            }
            ++remainders[remainder];
        }
        if (remainders[0] % 2 == 1) {
            return false;
        }
        for (int i = 1; i <= k / 2; ++i) {
            if (remainders[i] != remainders[k - i]) {
                return false;
            }
        }
        return true;
    }
};
```

Find the Longest Substring Containing Vowels in Even Counts [Leetcode 1371]

Given the string *s*, return the size of the longest substring containing each vowel an even number of times. That is, 'a', 'e', 'i', 'o', and 'u' must appear an even number of times.

Example 1:

Input: *s* = "eetminicoworoep"

Output: 13

Explanation: The longest substring is "leetminicowor" which contains two each of the vowels: e, i and o and zero of the vowels: a and u.

Example 2:

Input: *s* = "leetcodeisgreat"

Output: 5

Explanation: The longest substring is "leetc" which contains two e's.

Example 3:

Input: *s* = "bcbcbcb"

Output: 6

Explanation: In this case, the given string "bcbcbcb" is the longest because all vowels: a, e, i, o and u appear zero times.

Constraints:

- $1 \leq s.length \leq 5 \times 10^5$
- *s* contains only lowercase English letters.

Approach:

- The given intuition will give n^2 approach, so instead of that we thought of storing whether at that index frequency of each vowel is odd or even.
- So we used 5 bits to store whether the frequency is odd or even, which is initialised as 0 in decimal or "00000" in binary.
- Every time we encounter the same vowel we flip the bit corresponding to it.
- As we know that we will get answer when we reach to same state again for eg if we are on state "10010" we need to be in same state in future to have frequency of every element as even. (This point might be little hard to understand just go through this once again)

Complexity:

- **Time complexity:** $O(n)$
- **Space complexity:** $O(1)$ Constant extra space

Code:

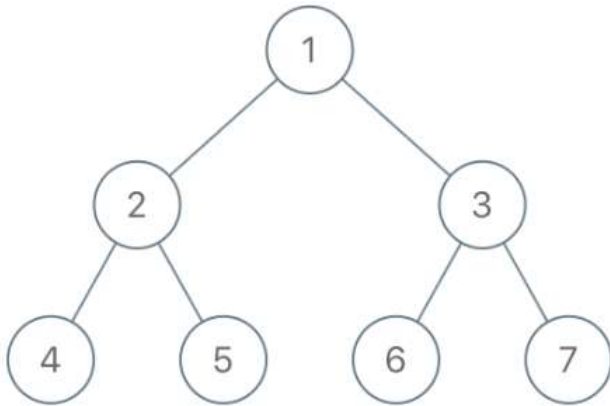
```
class Solution {
public:
    int findTheLongestSubstring(string s) {
        int ans=0;
        int num=0; // It stores current state at every index
        vector<int> v(32,-1); //For storing first index of that
state
        int i=0;
        for(auto a:s){
            if(a=='a')
                num = (num ^ (1 << 0)); //Code for flipping bit
            if(a=='e')
                num = (num ^ (1 << 1)); //Code for flipping bit
            if(a=='i')
                num = (num ^ (1 << 2)); //Code for flipping bit
            if(a=='o')
                num = (num ^ (1 << 3)); //Code for flipping bit
            if(a=='u')
                num = (num ^ (1 << 4)); //Code for flipping bit

            if(v[num]==-1){
                v[num]=i; // first encounter of state
            }
            else{
                ans=max(ans,i-v[num]); //second and after
encounters
            }
            if(num==0){
                ans=i+1; //becuase initially state or num is 0
            }
            i++;
        }
        return ans;
    }
};
```

Delete Nodes And Return Forest [Leetcode 1110]

Given the root of a binary tree, each node in the tree has a distinct value.
After deleting all nodes with a value in `to_delete`, we are left with a forest (a disjoint union of trees).
Return the roots of the trees in the remaining forest. You may return the result in any order.

Example 1:



Input: root = [1,2,3,4,5,6,7], to_delete = [3,5]

Output: [[1,2,null,4],[6],[7]]

Example 2:

Input: root = [1,2,4,null,3], to_delete = [3]

Output: [[1,2,4]]

Constraints:

- The number of nodes in the given tree is at most 1000.
- Each node has a distinct value between 1 and 1000.
- to_delete.length <= 1000
- to_delete contains distinct values between 1 and 1000.

Approach:

Consider a queue data structure --> to store all the valid Rootnodes which are not to be deleted

- First push root into queue and while loop exits when its empty(no nodes are left)
- del function as return type TreeNode* to link NULL to its parent
- Traverse from rootnode, If root->val is to be deleted store its left and right (if any) for further traversal and return NULL
- So for every recursion call it checks for delete node and returns root if no delete node is found
- rootnode is sorted in vector and queue is popped
- ans vector is returned when queue is empty

Complexity:

- **Time complexity:** $O(N)$ we are traversing each node only once
- **Space complexity:** $O(N)$ Assuming worst case where every node is to be deleted

Code:

```

/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x),
left(left), right(right) {}
 * };
 */
class Solution {
public:
    TreeNode* del(TreeNode*
root,queue<TreeNode*>&Rootnodes,vector<int>&to_delete){
        if(!root){
            return NULL;
        }
        if(find(to_delete.begin(),to_delete.end(),root->val)
!=to_delete.end()){
            if(root->left){
                Rootnodes.push(root->left);
            }
            if(root->right){
                Rootnodes.push(root->right);
            }
            return NULL;
        }
        root->left=del(root->left,Rootnodes,to_delete);
        root->right=del(root->right,Rootnodes,to_delete);
        return root;
    }
    vector<TreeNode*> delNodes(TreeNode* root, vector<int>&
to_delete) {
        vector<TreeNode*>ans;
        queue<TreeNode*>Rootnodes;
        Rootnodes.push(root);
        while(!Rootnodes.empty()){
            TreeNode* treeroot=Rootnodes.front();
            TreeNode* rootnode=del(treeroot,Rootnodes,to_delete);
            if(rootnode){
                ans.push_back(rootnode);
            }
            Rootnodes.pop();
        }
        return ans;
    }
};

```


What is a multimap?

In C++ STL, `std::multimap` is an associative container that is similar to `std::map` but allows multiple elements with the same key. It is part of the Standard Template Library and provides a way to associate multiple values with the same key.

Here are some key characteristics of `std::multimap`:

Key-Value Pair:

- Like `std::map`, each element in `std::multimap` is a key-value pair.

Duplicate Keys:

- Unlike `std::map`, `std::multimap` allows multiple elements with the same key. This means that there can be multiple values associated with a single key.

Ordered:

- The elements in a `std::multimap` are ordered based on the key values.

Implemented as Binary Search Tree:

- Internally, `std::multimap` is typically implemented as a binary search tree (e.g., red-black tree) to ensure efficient insertion, deletion, and search operations.

Usage:

- `std::multimap` is useful when you need to associate multiple values with the same key, and the order of elements based on keys is important.

Member Functions:

- `insert`: Inserts a key-value pair into the multimap.
- `erase`: Removes elements with a specific key or a range of elements.
- `find`: Searches for an element with a specific key.
- `count`: Returns the number of elements with a specific key.
- `begin`, `end`: Iterators to traverse the multimap.

Here's a simple example of using `std::multimap`:

Code:

```
#include <iostream>
#include <map>

int main() {
    std::multimap<int, std::string> myMultimap;

    // Inserting elements
    myMultimap.insert(std::make_pair(1, "One"));
    myMultimap.insert(std::make_pair(2, "Two"));
    myMultimap.insert(std::make_pair(1, "Uno")); // Allows
    multiple values with the same key
}
```

```
// Accessing elements
for (const auto& pair : myMultimap) {
    std::cout << pair.first << ": " << pair.second <<
std::endl;
}

return 0;
}
```

In this example, `std::multimap` is used to store key-value pairs, and it allows multiple values associated with the same key. The order is maintained based on the keys.

Largest Values From Labels [Leetcode 1090]

There is a set of n items. You are given two integer arrays `values` and `labels` where the value and the label of the i th element are `values[i]` and `labels[i]` respectively. You are also given two integers `numWanted` and `useLimit`. Choose a subset s of the n elements such that:

- The size of the subset s is less than or equal to `numWanted`.
- There are at most `useLimit` items with the same label in s .

The score of a subset is the sum of the values in the subset. Return the maximum score of a subset s .

Example 1:

Input: `values = [5,4,3,2,1]`, `labels = [1,1,2,2,3]`, `numWanted = 3`, `useLimit = 1`
Output: 9

Explanation: The subset chosen is the first, third, and fifth items.

Example 2:

Input: `values = [5,4,3,2,1]`, `labels = [1,3,3,3,2]`, `numWanted = 3`, `useLimit = 2`
Output: 12

Explanation: The subset chosen is the first, second, and third items.

Example 3:

Input: `values = [9,8,8,7,6]`, `labels = [0,0,0,1,1]`, `numWanted = 3`, `useLimit = 1`
Output: 16

Explanation: The subset chosen is the first and fourth items.

Constraints:

- $n == \text{values.length} == \text{labels.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{values}[i], \text{labels}[i] \leq 2 * 10^4$
- $1 \leq \text{numWanted}, \text{useLimit} \leq n$

Approach:

1. Take a vector of pairs (vct) to store values & their corresponding labels and a map to count how many elements of a particular label have been added to score.
2. Run a loop to store the values and labels in our vct.
3. Sort vct in descending order (to maximize score) based on the first element (values) from the pair then iterate through vct.
4. Check if the current element's label has been used already to the useLimit. If yes then skip.
5. If it's under the useLimit, add the value to the score. Increase the label's usage frequency by 1 and decrease numWanted by 1.
6. If numWanted has reached zero then our requirement has been satisfied, we can exit the loop.
7. Return the score.

Complexity:

- **Time complexity:**

Sorting : $O(n \log n)$

Iterating : $O(n)$

- **Space complexity:**

$O(n)$

Code:

```
class Solution {
public:
    int largestValsFromLabels(vector<int>& values, vector<int>&
labels, int numWanted, int useLimit) {
        vector<pair<int,int>> vct; //to store values & their
corresponding labels in pairs
        map<int,int> mpp;          // to count how many elements
of a particular label have
                                   // been added to score
        int score=0;               // maximum score of the
subset;

        //pairing up the values with their labels
        for(int i=0; i<values.size(); i++){
            vct.push_back(make_pair(values[i],labels[i]));
        }

        //sorting the pairs w.r.t. values in descending order
        sort(vct.begin(),vct.end(),greater<pair<int,int>>());

        // iterating through each pair
        for(int i=0; i<vct.size(); i++){
            // check whether the current pair's label has already
already been used more
            // than the permitted useLimit
```

```

        // if not then
        if(!mpp[vct[i].second] || mpp[vct[i].second] <
useLimit){
            score += vct[i].first;           //add the value of
score
            mpp[vct[i].second]++;           //add 1 to label's
freq of being used in score
            numWanted--;                     //decrease numbers
needed for subset score
        }
        if(numWanted==0) break;             // requirement
        satisfied hence end loop
    }
    return score;
}
};

```

Number of Substrings Containing All Three Characters [Leetcode 1358]

Given a string s consisting only of characters a , b and c .

Return the number of substrings containing at least one occurrence of all these characters a , b and c .

Example 1:

Input: $s = \text{"abcabc"}$

Output: 10

Explanation: The substrings containing at least one occurrence of the characters a , b and c are "abc", "abca", "abcab", "abcabc", "bca", "bcab", "bcabc", "cab", "cabc" and "abc" (again).

Example 2:

Input: $s = \text{"aaacb"}$

Output: 3

Explanation: The substrings containing at least one occurrence of the characters a , b and c are "aaacb", "aacb" and "acb".

Example 3:

Input: $s = \text{"abc"}$

Output: 1

Constraints:

- $3 \leq s.length \leq 5 \times 10^4$
- s only consists of a , b or c characters.

Approach:

1. Initialize an unordered map to keep track of the count of each character in the current window, count variable to store the number of valid substrings, start and end pointers, and the length of the string n.
2. Move the end pointer to the right until we find a window that contains all three characters 'a', 'b', and 'c'. For this, we can use a while loop that checks if the count of each character in the map is greater than zero.
3. Once we find a valid window, we know that all the substrings starting from the start pointer up to the end pointer will contain all three characters. We can add the count of such substrings to our answer by adding $n - \text{end}$ to the count variable. This is because we can form $n - \text{end}$ substrings from the current end pointer to the end of the string.
4. Move the start pointer to the right until we no longer have a valid window. We do this by decrementing the count of the character at the start pointer in the map and moving the start pointer to the right until we no longer have all three characters in the window.
5. Repeat steps 2 to 4 until we reach the end of the string.
6. Return the count variable.

Complexity:

- **Time complexity:** $O(n)$ where n is the length of the string. We visit each character of the string at most twice, once by the end pointer and once by the start pointer.
- **Space complexity:** $O(1)$ since the size of the unordered map is constant at 3 (for the characters 'a', 'b', and 'c').

Code:

```
class Solution {
public:
    int numberOfSubstrings(string s) {
        unordered_map<char,int> map;
        int count = 0, start = 0, end = 0;
        int n = s.size();
        while(end<n){
            map[s[end]]++;
            while( map['a'] && map['b'] && map['c'] ){
                count+=n-end;
                map[s[start]]--;
                start++;
            }
            end++;
        }
        return count;
    }
};
```

Number of Submatrices That Sum to Target [Leetcode 1074]

Given a matrix and a target, return the number of non-empty submatrices that sum to target.

A submatrix $x1, y1, x2, y2$ is the set of all cells $\text{matrix}[x][y]$ with $x1 \leq x \leq x2$ and $y1 \leq y \leq y2$.

Two submatrices $(x1, y1, x2, y2)$ and $(x1', y1', x2', y2')$ are different if they have some coordinate that is different: for example, if $x1 \neq x1'$.

Example 1:

0	1	0
1	1	1
0	1	0

Input: matrix = `[[0,1,0],[1,1,1],[0,1,0]]`, target = 0

Output: 4

Explanation: The four 1x1 submatrices that only contain 0.

Example 2:

Input: matrix = `[[1,-1],[-1,1]]`, target = 0

Output: 5

Explanation: The two 1x2 submatrices, plus the two 2x1 submatrices, plus the 2x2 submatrix.

Example 3:

Input: matrix = `[[904]]`, target = 0

Output: 0

Constraints:

- $1 \leq \text{matrix.length} \leq 100$
- $1 \leq \text{matrix}[0].\text{length} \leq 100$
- $-1000 \leq \text{matrix}[i] \leq 1000$
- $-10^8 \leq \text{target} \leq 10^8$

Approach:

For each pair of columns (i, j) from index i to j inclusive:

Create a hashmap (freq) to store the frequency of cumulative sums. Initialize the cumulative sum sum as 0. Initialize cumuColSum to track the cumulative column sums for each row, starting from column i to column j. Traverse each row (k) from 0 to n-1:

Update the cumuColSum[k] with the current value in matrix[k][j] plus the previous cumulative sum value. Increment the frequency of the current cumulative sum sum in the freq map. Update sum by adding the current cumuColSum[k]. At each step in row traversal:

Increment the count by the frequency of (sum - target) in the freq map. This implies that there are freq[sum - target] submatrices that sum to the target value ending at the current row. The count is the final result representing the total number of submatrices across all column pairs that sum up to the target value.

Complexity:

- **Time complexity:** $O(m^2 \cdot n)$
- **Space complexity:** $O(m \cdot n)$

Code:

```
class Solution {
public:
    int numSubmatrixSumTarget(vector<vector<int>>& matrix, int
target) {
        int n = matrix.size();
        int m = matrix[0].size();

        int count =0;

        for(int i=0;i<m;i++){
            vector<int> cumuColSum(n,0);

            for(int j=i;j<m;j++){
                unordered_map<int,int> freq;
                int sum = 0;
                for(int k=0;k<n;k++){
                    cumuColSum[k] = matrix[k][j] + cumuColSum[k];
                    freq[sum]++;
                    sum += cumuColSum[k];
                    count += freq[sum-target];
                }
            }
        }

        return count;
    }
};
```



**THANK
YOU !**

